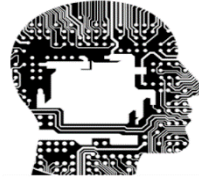




Université Constantine 2
جامعة قسنطينة 2



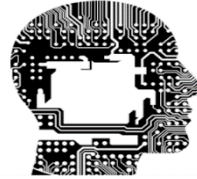
Apprentissage par Machine 02

APM2

Dr. NECIBI Khaled
New Technologies Faculty
Khaled.necibi@univ-constantine2.dz



Université Constantine 2
جامعة قسنطينة 2



Apprentissage par Machine APM 02

Workshop 03

Dr. NECIBI Khaled

New Technologies Faculty

Khaled.necibi@univ-constantine2.dz

Etudiants concernés

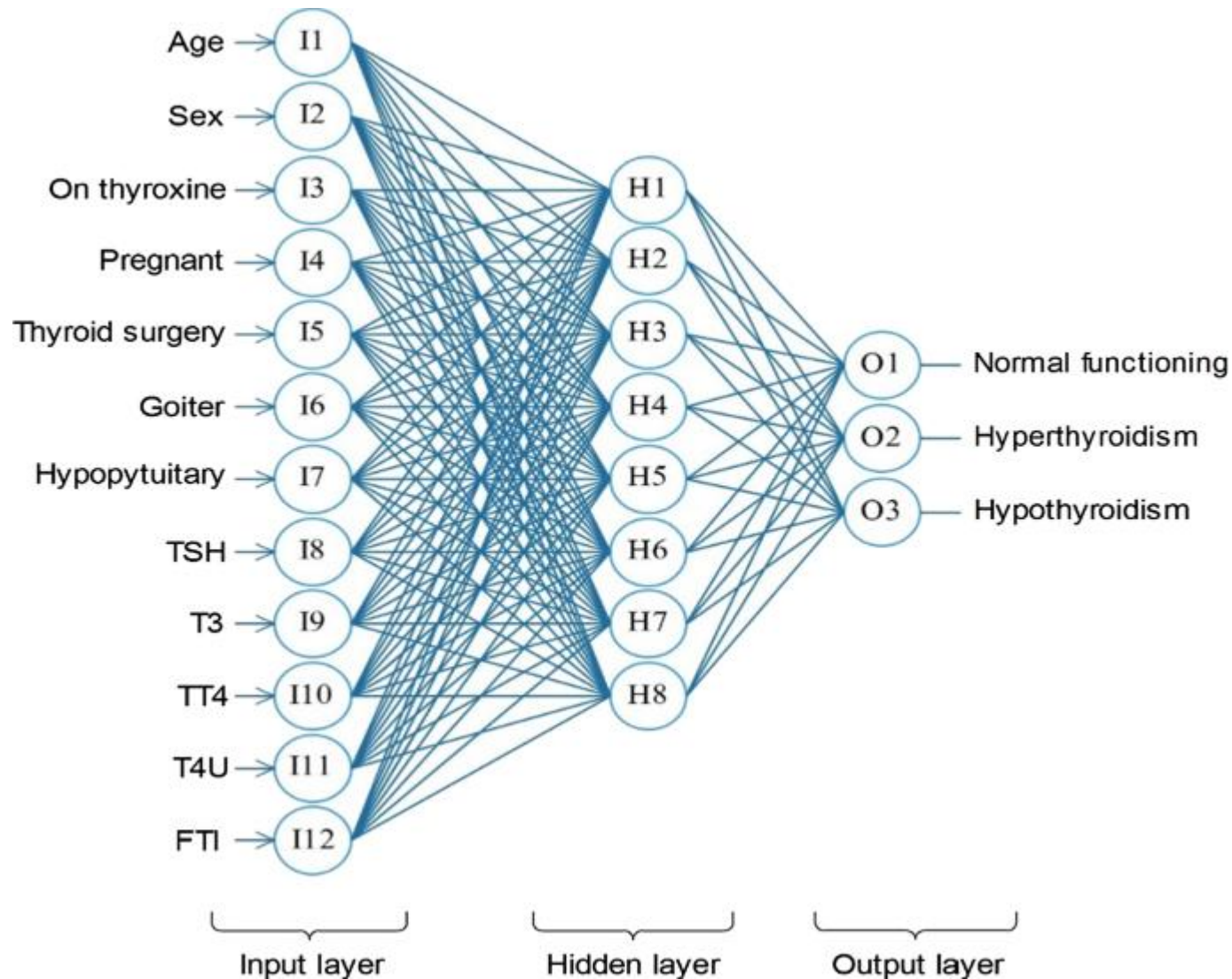
Faculté/Institut	Département	Niveau	Spécialité
New Technologies	IFA	Master 01	STIC

Simple Neural Network Architecture

● Multi-Layer Perceptron (MLP)

- An MLP is a type of feedforward artificial neural network composed of multiple layers of neurons
- It is one of the simplest forms of deep learning models
- Widely used for classification and regression tasks
- The MLP Consists of:
 - **Input layer:** receives input features (e.g., pixel values of an image)
 - **Hidden layers:** perform non-linear transformations using activation functions
 - **Output layer:** Produces the final classification result

Simple Neural Network Architecture



Simple Neural Network Architecture

- **MLP for image classification?**
 - MLP can learn complex patterns in data through multiple layers of neurons
 - It is effective for simple image classification tasks (e.x., flattened image inputs)
 - MLP are computationally lighter than other deep architectures (Convolutional neural networks)
 - MLP works well for small-scale image datasets

Simple Neural Network Architecture

- **MLP for image classification?**
 - **Input processing:** images are flattened into a 1D vector before feeding into the network
 - **Feature extraction:** Learns global patterns but loses local spatial relationships
 - Best for small datasets, structured data, and basic image tasks
 - **Computational cost:** less demanding comparing to other complex architecture

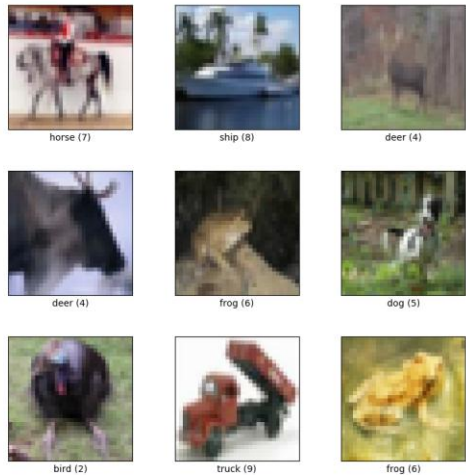
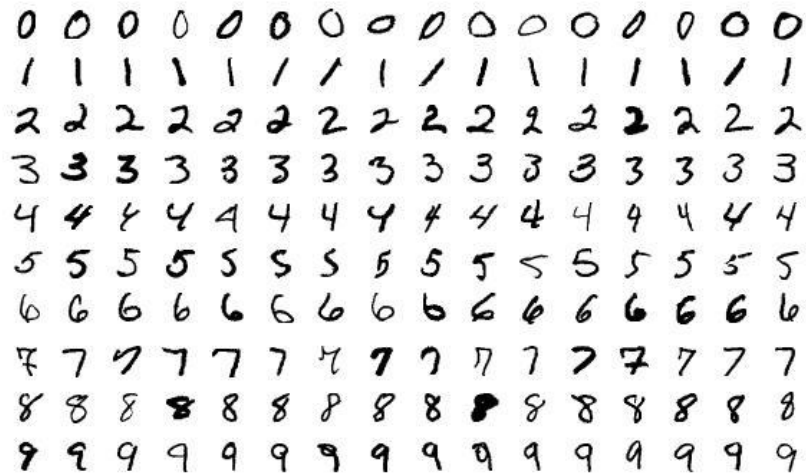
Simple Neural Network Architecture

- **Preprocessing image data for MLP**
 - Before feeding image data into an MLP, we need to:
 - Convert,
 - Normalize, and
 - Structure the data properly
 - Unlike CNN, which works with 2D image structure,
 - MLP require flattened input vectors
 - Raw images cannot be directly used in an MLP; they must be converted into numerical values
 - Image data often has different scales, noise, and irrelevant details that can degrade model performance
 - Proper preprocessing helps
 - Improve training efficiency
 - Enhance model accuracy
 - Reduce computational cost

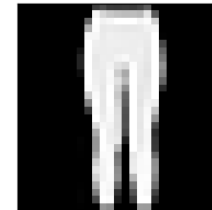
Simple Neural Network Architecture

- **Preprocessing – Loading image data**
 - Common datasets for simple image classification
 - MNIST (handwritten digits, 28×28 grayscale images) (Groupe 02)
 - Fashion-MNIST (clothing images, 28×28 grayscale) (Groupe 03)
 - CIFAR-10 (color images, 32×32 RGB) (Groupe 01)
 - These datasets can be loaded using TensorFlow/Keras or PyTorch
- **Preprocessing – Flattening the image data**
 - MLP requires 1D vectors as input (not 2D images)
 - Flatten each (height × width) image into a 1D vector
 - For MNIST 28x28 images:
 - Each image has 784 pixels (28×28)
 - Convert each image from (28,28) → (784,1)

Simple Neural Network Architecture



Pullover (2)



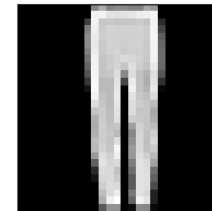
Trouser (1)



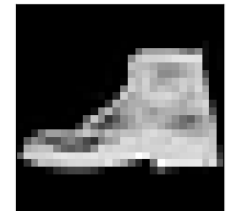
Bag (8)



Coat (4)



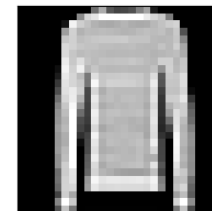
Trouser (1)



Ankle boot (9)



Pullover (2)



Pullover (2)



T-shirt/top (0)

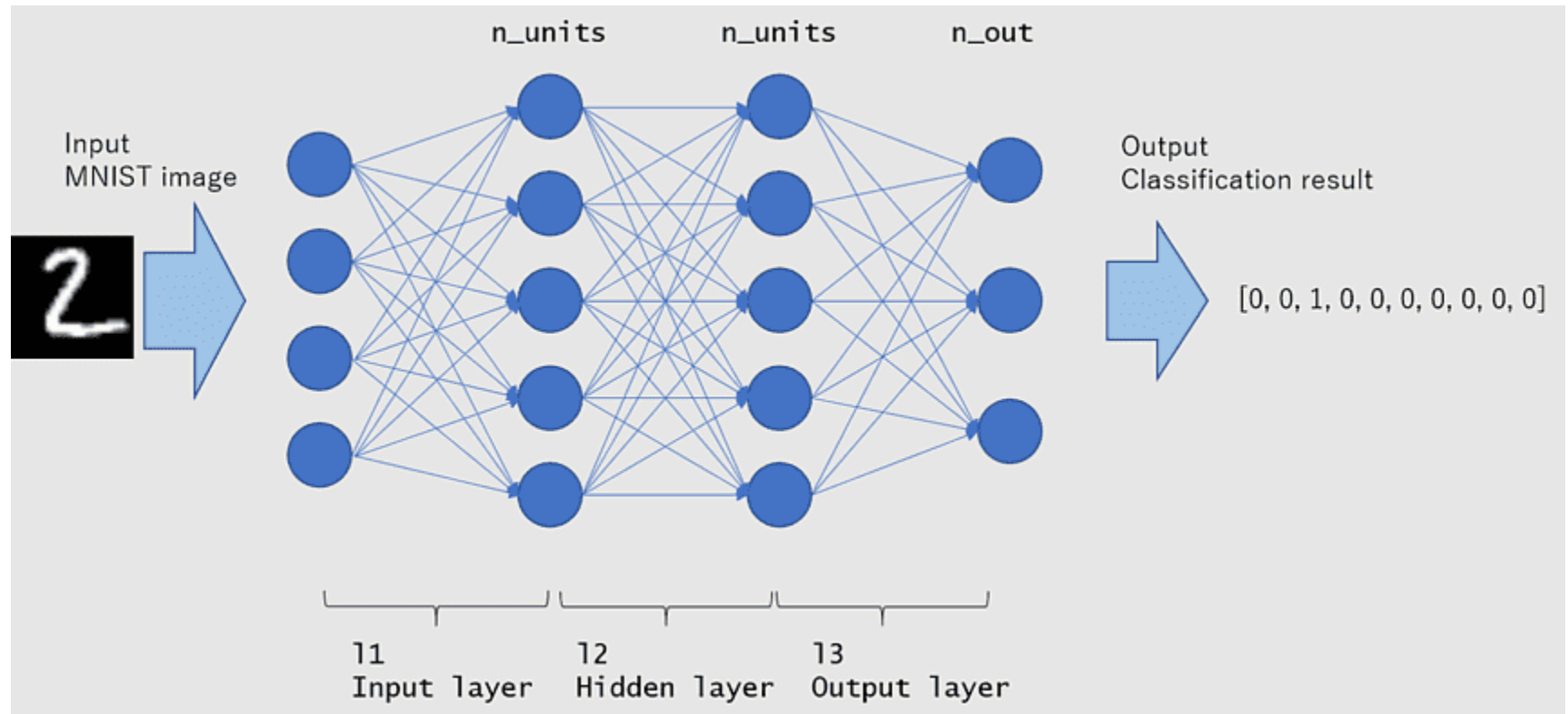
Simple Neural Network Architecture

```
ModelPerformance.py U  training_after_data_augm.ipynb  MLPWorkshop 4, U X
Code > MLPWorkshop > ...
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5  from tensorflow.keras.datasets import mnist
6  from tensorflow.keras.models import Sequential
7  from tensorflow.keras.layers import Dense
8  from tensorflow.keras.utils import to_categorical
9  from sklearn.metrics import classification_report, confusion_matrix
```

Simple Neural Network Architecture

- **Preprocessing – Normalizing the image data**
 - Pixel values range from 0 to 255
 - Normalization (scaling between 0 and 1) improves training stability
 - Faster convergence
 - Better numerical stability
 - Avoids large weight updates
- **Preprocessing – One-Hot encoding labels**
 - Classification labels must be converted into a one-hot encoded format
 - Example for MNIST (digits 0-9 → 10 classes):
 - Avoids treating labels as ordinal numbers
 - Better suited for softmax output in MLP
- **Preprocessing – Splitting data into training & validation**
 - 80% Training, 20% Testing is a common split
 - Prevents overfitting, and allows performance tuning before final evaluation

Simple Neural Network Architecture



Simple Neural Network Architecture

```
ModelPerformance.py U  training_after_data_augm.ipynb  MLPWorkshop 4, U  SLP
```

```
Code > MLPWorkshop > ...
```

```
12 (x_train, y_train), (x_test, y_test) = mnist.load_data()
13
14 x_train = x_train.reshape(x_train.shape[0], 28*28).astype('float32') / 255.0
15 x_test = x_test.reshape(x_test.shape[0], 28*28).astype('float32') / 255.0
16
17 y_train = to_categorical(y_train, 10)
18 y_test_categorical = to_categorical(y_test, 10)
```

Simple Neural Network Architecture

- MLP Architecture for image classification
 - Input layer:
 - Accepts flattened image data
 - Example: a 28×28 image (MNIST) is reshaped into a 784 dimensional vector
 - Hidden layers:
 - Each hidden layer contains neurons that apply activation functions (e.g., ReLU, Sigmoid)
 - These layers learn patterns from image pixels
 - More hidden layers → better feature extraction
 - Output layer:
 - Produces the final classification
 - Uses **softmax** activation for multi-class classification

Simple Neural Network Architecture

- MLP Architecture for MNIST (handwritten digits)
 - Input layer
 - 784 neurons (28x28 pixels flattened)
 - Hidden layers1
 - 128 neurons ReLU activation
 - Hidden layers 2
 - 64 neurons, ReLU activation
 - Output layer
 - 10 neurons (one per digit), Softmax activation

Simple Neural Network Architecture

```
ModelPerformance.py U  training_after_data_augm.ipynb  MLPWorkshop 4, U ●
Code > MLPWorkshop > ...
23 model = Sequential([
24     Dense(128, activation='relu', input_shape=(784,)),
25     Dense(64, activation='relu'),
26     Dense(10, activation='softmax')
27 ])
```


Simple Neural Network Architecture

- Loss function & optimization for MLP
 - Categorical Cross-Entropy
 - It penalizes incorrect predictions more heavily
 - Encourages the network to produce high confidence predictions
 - $L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$
 - C : total number of classes
 - y_i : true labels (one-hot encoded, either 0 or 1)
 - $\hat{y}_i$: predicted probability for class i
- Optimizers in MLP (most popular)
 - SGD
 - Adam
 - ...

Simple Neural Network Architecture

ModelPerformance.py U

training_after_data_augm.ipynb

MLPWorkshop 4, U ●

SLP

U ●

Code > MLPWorkshop > ...

```
30 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Simple Neural Network Architecture

- **Training the MLP model**
 - Now that we have defined the loss function and optimizer, we can train the model:
 - **Forward Propagation** → prediction over input data
 - **Loss computation** → difference between predictions and actual labels
 - **Backward propagation** → update model weights using gradient descent (or other technique)
 - **Iterations** → the process repeat over multiple epochs until convergence

Simple Neural Network Architecture

ModelPerformance.py U

training_after_data_augm.ipynb

MLPWorkshop 4, U

SLP U

Code > MLPWorkshop > ...

```
33 history = model.fit(x_train, y_train, epochs=10, batch_size=32, validation_data=(x_test, y_test_categorical))
```

```
Epoch 1/10
1875/1875 ————— 23s 10ms/step - accuracy: 0.8795 - loss: 0.4176 - val_accuracy: 0.9614 - val_loss: 0.1287
Epoch 2/10
1875/1875 ————— 17s 9ms/step - accuracy: 0.9656 - loss: 0.1124 - val_accuracy: 0.9739 - val_loss: 0.0874
Epoch 3/10
1875/1875 ————— 16s 8ms/step - accuracy: 0.9789 - loss: 0.0699 - val_accuracy: 0.9731 - val_loss: 0.0866
Epoch 4/10
1875/1875 ————— 15s 8ms/step - accuracy: 0.9836 - loss: 0.0521 - val_accuracy: 0.9773 - val_loss: 0.0721
Epoch 5/10
1875/1875 ————— 15s 8ms/step - accuracy: 0.9872 - loss: 0.0402 - val_accuracy: 0.9756 - val_loss: 0.0841
Epoch 6/10
1875/1875 ————— 15s 8ms/step - accuracy: 0.9901 - loss: 0.0329 - val_accuracy: 0.9800 - val_loss: 0.0715
Epoch 7/10
1875/1875 ————— 9s 5ms/step - accuracy: 0.9912 - loss: 0.0272 - val_accuracy: 0.9811 - val_loss: 0.0707
Epoch 8/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9933 - loss: 0.0213 - val_accuracy: 0.9805 - val_loss: 0.0774
Epoch 9/10
1875/1875 ————— 6s 3ms/step - accuracy: 0.9939 - loss: 0.0181 - val_accuracy: 0.9773 - val_loss: 0.0953
Epoch 10/10
1875/1875 ————— 11s 4ms/step - accuracy: 0.9950 - loss: 0.0151 - val_accuracy: 0.9770 - val_loss: 0.0850
```

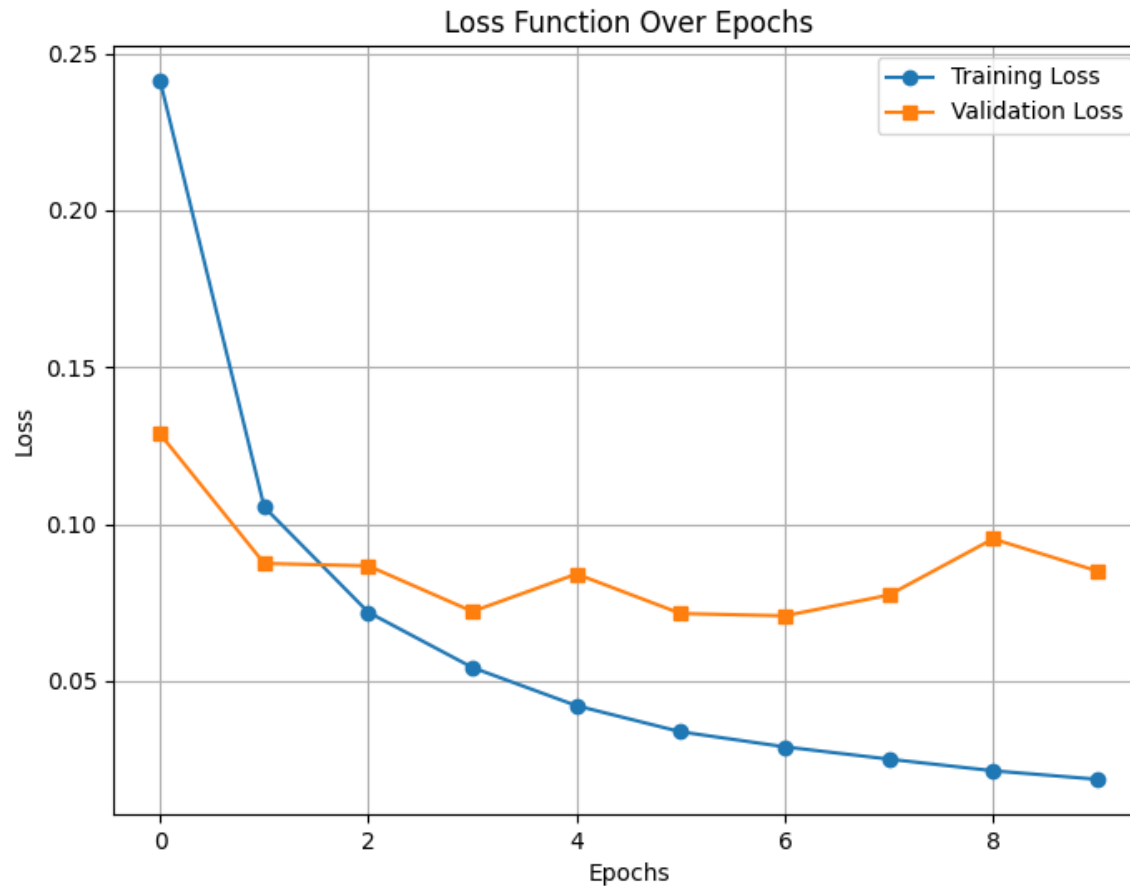
Simple Neural Network Architecture

- Plot the loss function over Epochs

```
ModelPerformance.py U  training_after_data_augm.ipynb  MLPWorkshop 4, U  SLP
Code > MLPWorkshop > ...
36 plt.figure(figsize=(8, 6))
37 plt.plot(history.history['loss'], label='Training Loss', marker='o')
38 plt.plot(history.history['val_loss'], label='Validation Loss', marker='s')
39 plt.xlabel('Epochs')
40 plt.ylabel('Loss')
41 plt.title('Loss Function Over Epochs')
42 plt.legend()
43 plt.grid()
44 plt.show()
```

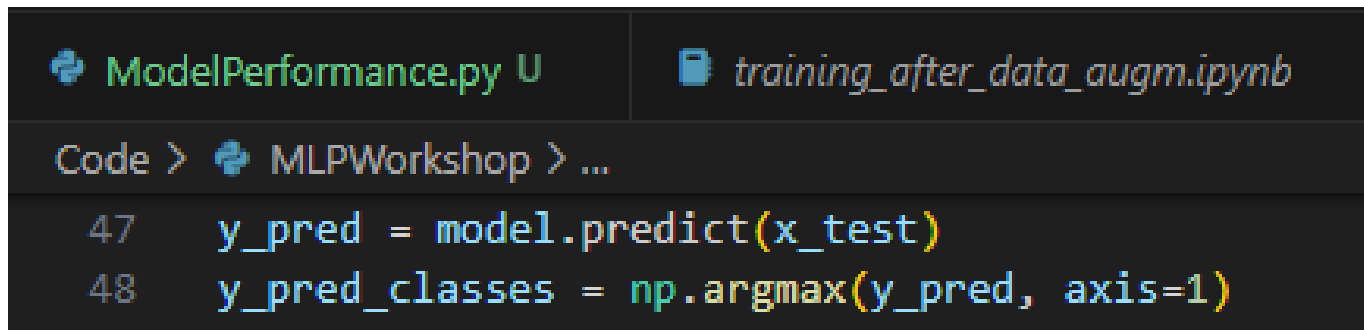
Simple Neural Network Architecture

- Plot the loss function over Epochs



Simple Neural Network Architecture

- Model Evaluation and Prediction



The screenshot shows a Jupyter Notebook with two tabs: 'ModelPerformance.py U' and 'training_after_data_augm.ipynb'. The active tab is 'training_after_data_augm.ipynb'. Below the tabs, the breadcrumb 'Code > MLPWorkshop > ...' is visible. The code cell contains two lines of Python code:

```
47 y_pred = model.predict(x_test)
48 y_pred_classes = np.argmax(y_pred, axis=1)
```

Simple Neural Network Architecture

- Generate Classification Report

```
ModelPerformance.py U  training_after_data_augm.ipynb  MLPWorkshop 4, U ●
Code > MLPWorkshop > ...
51 print("\nClassification Report:\n")
52 print(classification_report(y_test, y_pred_classes))
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE <u>TERMINAL</u> PORTS					
	precision	recall	f1-score	support	
0	0.99	0.98	0.99	980	
1	0.99	0.99	0.99	1135	
2	0.97	0.99	0.98	1032	
3	0.98	0.97	0.97	1010	
9	0.97	0.97	0.97	1009	
accuracy			0.98	10000	
macro avg	0.98	0.98	0.98	10000	
weighted avg	0.98	0.98	0.98	10000	

Simple Neural Network Architecture

- Plot Confusion Matrix

```
ModelPerformance.py U  training_after_data_augm.ipynb  MLPWorkshop 4, U  SLP  U  ●
Code > MLPWorkshop > ...
55 cm = confusion_matrix(y_test, y_pred_classes)
56 plt.figure(figsize=(8,6))
57 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=range(10), yticklabels=range(10))
58 plt.xlabel('Predicted Label')
59 plt.ylabel('True Label')
60 plt.title('Confusion Matrix')
61 plt.show()
```

Simple Neural Network Architecture

- Plot Confusion Matrix

